

Tema 08 · Servlets: web.xml, anotaciones, forward y MVC

Clase 022 (16-09) · Teoría: «Servlets» (Clase023_Applet_Servlet.pdf)

Programación Avanzada · Prof. Macrino Matías · Universidad del Salvador – Sede Pilar
· 2.º cuatrimestre

1. Qué es un Servlet

Un **Servlet** es una clase Java que se ejecuta **en el servidor** y atiende peticiones HTTP: recibe la petición del cliente y devuelve una respuesta (normalmente HTML, pero puede ser JSON, un PDF, una imagen..., cualquier **tipo MIME**). Vive dentro de un **contenedor web** (Tomcat), que se encarga de recibir las peticiones, crear los objetos y manejar los hilos. Para programarlos se usa el paquete `javax.servlet.http` (Tomcat 9) o `jakarta.servlet.http` (Tomcat 10+).

Se crean **extendiendo `HttpServlet`** y sobrescribiendo **`doGet`** (peticiones GET) y/o **`doPost`** (peticiones POST). Ambos reciben dos objetos:

Objeto	Para qué sirve	Métodos usuales
<code>HttpServletRequest</code> (req)	La petición: todo lo que envía el cliente	<code>getParameter("x")</code> (datos del formulario o de la URL), <code>getParameterValues</code> , <code>getSession()</code> , <code>getCookies()</code> , <code>getMethod()</code> , <code>getQueryString()</code> , <code>getRemoteAddr()</code> , <code>setAttribute /</code> <code>getAttribute</code> , <code>getRequestDispatcher()</code>
<code>HttpServletResponse</code> (resp)	La respuesta: lo que se devuelve	<code>setContentType("text/html")</code> , <code>getWriter()</code> (<code>PrintWriter</code> para escribir), <code>setStatus(int)</code> , <code>sendRedirect(url)</code> , <code>addCookie()</code> , <code>sendError()</code>

2. Ciclo de vida

Fase	Método	Cuándo ocurre y para qué
Inicialización	<code>init()</code>	Una sola vez , cuando el contenedor crea la instancia (en la primera petición o al arrancar). Se usa para cargar configuración, crear DAO, abrir recursos.
Servicio	<code>service()</code> → <code>doGet</code> / <code>doPost</code> / <code>doPut</code> / <code>doDelete</code>	En cada petición . <code>service()</code> mira el método HTTP y llama al <code>doXxx</code> correspondiente.
Destrucción	<code>destroy()</code>	Una sola vez , antes de descargar el servlet. Se usa para cerrar conexiones, guardar datos, liberar recursos.

Multihilo: el contenedor crea **una única instancia** de cada servlet y, por cada petición, un **hilo** que ejecuta `service()`. Es decir: N peticiones simultáneas = N hilos sobre el mismo objeto. Consecuencia: **no se deben guardar datos específicos de una petición o de un usuario en atributos de la clase del servlet** (los compartirían todos los hilos); esos datos van en variables locales, en el request o en la session. Los atributos de instancia son seguros solo si son de sólo lectura o están sincronizados (como el DAO inicializado en `init()`).

3. Declaración y mapeo: web.xml vs. anotaciones

Hay que decirle al contenedor **qué clase es un servlet y en qué URL responde**. Dos formas (el parcial pide conocer ambas):

	web.xml (descriptor)	Anotación @WebServlet
Cómo	Dos bloques: <servlet> (nombre + clase) y <servlet-mapping> (mismo nombre + url-pattern)	Una anotación sobre la clase: @WebServlet("/productos") o @WebServlet(name="x", urlPatterns="/x")
Ventajas	Configuración centralizada, se cambia sin recompilar	Más corto; la URL está junto al código (se mantiene mejor)
Requisito	—	Servlet 3.0+ (en web.xml usar version 3.0 o superior)

```
<!-- web.xml -->
<servlet>
  <servlet-name>HolaMundoServlet</servlet-name>
  <servlet-class>edu.usal.servlet.PrimerServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>HolaMundoServlet</servlet-name>
  <url-pattern>/hola-mundo</url-pattern>
</servlet-mapping>

// Equivalente con anotación
@WebServlet(name = "HolaMundoServlet", urlPatterns = "/hola-mundo")
public class PrimerServlet extends HttpServlet { ... }
```

La URL final es **http://host:puerto/CONTEXTO/url-pattern**. El contexto es el nombre del WAR (clase-16-09), por eso el servlet queda en http://localhost:8080/clase-16-09/hola-mundo.

4. Respuestas: tipo MIME, estado y redirección

- **Tipo MIME** (setContentType) le dice al navegador cómo interpretar la respuesta: text/html, text/plain, application/json, application/pdf, application/zip, image/jpeg. Con texto en español hay que indicar la codificación: "text/html; charset=UTF-8".
- **Código de estado** (setStatus, sendError): 200 OK, 302 redirección, 400 solicitud incorrecta, 403 prohibido, 404 no encontrado, 405 método no permitido, 500 error del servidor.
- Si no se sobrescribe **doPost** y se recibe un POST, el comportamiento por defecto de HttpServlet es responder un **error (405, método no permitido)**: por eso llamar a super.doPost() en un servlet «vacío» equivale a no aceptar POST.

5. forward vs. sendRedirect

	RequestDispatcher.forward(req, resp)	response.sendRedirect(url)
Dónde ocurre	Dentro del servidor: se transfiere el control a otro recurso (un JSP)	El servidor responde 302 y el navegador hace una nueva petición a esa URL
URL del navegador	No cambia (el cliente no se entera)	Cambia a la nueva URL
Datos del request	Se conservan: se pueden pasar objetos con setAttribute	Se pierden (es otra petición)
Eficiencia	Más eficiente (una sola petición)	Dos viajes de red
Uso típico	Servlet que prepara datos y los muestra en un JSP (MVC)	Después de un POST (patrón Post/Redirect/Get) o para ir a otro sitio
Alcance	Solo recursos de la misma aplicación	Cualquier URL

6. Servlets + JSP = MVC

La arquitectura **MVC (Modelo-Vista-Controlador)** separa la aplicación en tres partes: el **Modelo** (datos y reglas: clases de dominio y DAO), la **Vista** (lo que se muestra: JSP + JSTL) y el **Controlador** (el servlet, que recibe la petición, pide datos al modelo y elige la vista). Flujo de esta clase:

```
Navegador --GET /productos--> ProductosServlet (Controlador)
  1. productosDAO.getAll()      <- Modelo
  2. req.setAttribute("listado", productos) <- deja los datos en el request
  3. forward a /WEB-INF/productos/lista-productos.jsp <- Vista
lista-productos.jsp recorre ${requestScope.listado} con JSTL y devuelve el HTML
```

Las vistas se ponen dentro de **WEB-INF**: esa carpeta **no es accesible directamente** desde el navegador, así que la única forma de ver la JSP es pasando por el servlet (que garantiza que los datos estén cargados). Es la práctica que usa la solución del parcial.

Applets: la teoría de esta misma clase (Clase023_Applet_Servlet.pdf) trata también los applets (programas Java dentro de una página web: init, start, stop, destroy, paint). Es una tecnología obsoleta (los navegadores ya no la soportan) y no se utiliza en el parcial; por eso este tema se centra en los servlets.

Código de las clases, explicado línea por línea

Cada recuadro gris reproduce líneas reales del archivo (con su número de línea) y el recuadro verde de abajo explica qué hacen y por qué están escritas así.

Clase 022 - 16-09: primer servlet (web.xml), servlet con anotaciones y listado de productos con forward

Tres servlets: **PrimerServlet** (declarado en web.xml, escribe HTML directamente), **SegundoServlet** (declarado con anotación) y **ProductosServlet** (patrón MVC: usa un DAO en memoria y hace forward a una JSP con JSTL). URLs: /hola-mundo, /segundo-servlet y /productos.

022 - Clase 16-09 / pom.xml

Qué es y para qué sirve: Archivo de configuración de Maven (POM) de una aplicación web: coordenadas, dependencias y nombre del WAR.

Cómo defenderlo / qué decir: Es el POM que genera el arquetipo de Maven para aplicaciones web. Lo importante: **packaging war**, las APIs de Servlet/JSP como **provided** (las aporta el contenedor) y, si la página usa JSTL, esas librerías sí van en el WAR. Esta versión incluye JSTL.

```
1 | <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2 |   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
```

Líneas 1 a 2: Cabecera del POM: elemento raíz **project** con sus espacios de nombres XML (definen el formato válido del archivo).

```
3 |   <modelVersion>4.0.0</modelVersion>
```

Línea 3: Versión del modelo de POM (siempre 4.0.0).

```
4 |   <groupId>edu.usal</groupId>
5 |   <artifactId>clase-16-09</artifactId>
6 |   <packaging>war</packaging>
7 |   <version>1.0-SNAPSHOT</version>
8 |   <name>clase-16-09 Maven Webapp</name>
9 |   <url>http://maven.apache.org</url>
```

Líneas 4 a 9: Coordenadas del proyecto: **groupId** edu.usal (la organización), **artifactId** clase-16-09 (el nombre), **packaging war** (se empaqueta como aplicación web, un archivo .war que se despliega en Tomcat), versión 1.0-SNAPSHOT, nombre descriptivo y URL (datos del arquetipo maven-archetype-webapp).

```
10 |   <dependencies>
```

Línea 10: Abre la lista de dependencias (librerías que Maven descarga solo).

```
11 |     <dependency>
12 |       <groupId>junit</groupId>
13 |       <artifactId>junit</artifactId>
14 |       <version>3.8.1</version>
15 |       <scope>test</scope>
16 |     </dependency>
17 |
```

Líneas 11 a 17: **JUnit 3.8.1** con **scope test**: solo se usa para pruebas unitarias, no viaja dentro del WAR.

```
18 |     <dependency>
19 |       <groupId>javax.servlet</groupId>
20 |       <artifactId>javax.servlet-api</artifactId>
21 |       <version>4.0.1</version>
22 |       <scope>provided</scope>
23 |     </dependency>
24 |
```

Líneas 18 a 24: **javax.servlet-api 4.0.1** con **scope provided**: se necesita para compilar los servlets, pero en ejecución la provee el servidor (Tomcat 9), por eso no se incluye en el WAR.

```

25 | <dependency>
26 |   <groupId>javax.servlet.jsp</groupId>
27 |   <artifactId>javax.servlet.jsp-api</artifactId>
28 |   <version>2.3.3</version>
29 |   <scope>provided</scope>
30 | </dependency>
31 |

```

Líneas 25 a 31: javax.servlet.jsp-api 2.3.3, también **provided**: la API de JSP que aporta Tomcat.

```

32 | <dependency>
33 |   <groupId>javax.servlet</groupId>
34 |   <artifactId>jstl</artifactId>
35 |   <version>1.2</version>
36 | </dependency>
37 |

```

Líneas 32 a 37: jstl 1.2 (scope compile por defecto): la implementación de JSTL; **Tomcat no la trae**, por eso debe ir dentro del WAR (carpeta WEB-INF/lib).

```

38 | <dependency>
39 |   <groupId>javax.servlet.jsp.jstl</groupId>
40 |   <artifactId>jstl-api</artifactId>
41 |   <version>1.2</version>
42 | </dependency>
43 |

```

Líneas 38 a 43: jstl-api 1.2: la API (interfaces) de JSTL.

```

44 | <dependency>
45 |   <groupId>>taglibs</groupId>
46 |   <artifactId>standard</artifactId>
47 |   <version>1.1.2</version>
48 | </dependency>

```

Líneas 44 a 48: taglibs standard 1.1.2: las librerías de etiquetas estándar (los .tld con core, fmt, etc.).

```

49 | </dependencies>

```

Línea 49: Cierre de la lista de dependencias.

```

50 | <build>
51 |   <finalName>clase-16-09</finalName>
52 | </build>

```

Líneas 50 a 52: finalName define el nombre del WAR generado (clase-16-09.war) y por lo tanto la **ruta de contexto** de la aplicación: http://localhost:8080/clase-16-09/.

```

53 | </project>

```

Línea 53: Cierre del POM.

022 - Clase 16-09 / src / main / webapp / WEB-INF / web.xml

Qué es y para qué sirve: web.xml: descriptor de despliegue de la aplicación web. Vive en WEB-INF y es el archivo de configuración que lee el contenedor (Tomcat) al iniciar.

Cómo defenderlo / qué decir: Es el descriptor clásico: los servlets se declaran con **<servlet>** (nombre + clase) y se mapean a una URL con **<servlet-mapping>** (mismo nombre + url-pattern). Es la alternativa a la anotación @WebServlet; el parcial pide conocer **ambas**.

```

1 | <!DOCTYPE web-app PUBLIC
2 |   "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3 |   "http://java.sun.com/dtd/web-app_2_3.dtd" >
4 |

```

Líneas 1 a 4: Declaración del tipo de documento (**DOCTYPE**): indica que es un descriptor de despliegue de aplicación web.

```

5 | <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
6 |     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
7 |     xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
8 |         http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
9 |     version="4.0">

```

Líneas 5 a 9: Elemento raíz **web-app** con los espacios de nombres de Java EE y **version="4.0"** (Servlet 4.0); habilita funciones modernas como las **anotaciones** (@WebServlet) y el EL de JSP 2.3.

```

10 | <display-name>Archetype Created Web Application</display-name>
11 |

```

Líneas 10 a 11: Nombre descriptivo de la aplicación (lo muestra el administrador de Tomcat).

```

12 | <!-- 1 - Declarar el Servlet -->
13 | <servlet>
14 |     <servlet-name>HolaMundoServlet</servlet-name>
15 |     <servlet-class>edu.usal.servlet.PrimerServlet</servlet-class>
16 | </servlet>
17 |

```

Líneas 12 a 17: Paso 1 - declarar el servlet: **servlet-name** es un nombre interno (el que se usa para vincularlo con su mapeo) y **servlet-class** es la clase completa (edu.usal.servlet.PrimerServlet).

```

18 | <!-- 2 - Mapeo de la URL de acceso -->
19 | <servlet-mapping>
20 |     <servlet-name>HolaMundoServlet</servlet-name>
21 |     <url-pattern>/hola-mundo</url-pattern>
22 | </servlet-mapping>
23 |

```

Líneas 18 a 23: Paso 2 - mapear la URL: **servlet-mapping** vincula el mismo servlet-name con el **url-pattern** /hola-mundo: la dirección en la que responde.

```

24 | </web-app>

```

Línea 24: Cierre del descriptor.

022 - Clase 16-09 / src / main / java / edu / usal / model / Productos.java

Qué es y para qué sirve: Clase de modelo (JavaBean): un producto con id, nombre y precio.

Cómo defenderlo / qué decir: Entidad (POJO) del patrón DAO: atributos privados, constructor vacío y con parámetros, getters y setters. Los getters son los que EL usa en la JSP (\${produ.nombre}).

```

1 | package edu.usal.model;
2 |

```

Líneas 1 a 2: Paquete del modelo.

```

3 | public class Productos {
4 |
5 |     private int id;
6 |     private String nombre;
7 |     private double precio;
8 |

```

Líneas 3 a 8: Atributos privados: id, nombre y precio.

```

9 |     public Productos() {
10 |     }
11 |

```

Líneas 9 a 11: Constructor vacío (útil para frameworks y para crear el objeto y completar con setters).

```

12 |     public Productos(int id, String nombre, double precio) {
13 |         this.id = id;
14 |         this.nombre = nombre;
15 |         this.precio = precio;
16 |     }
17 |

```

Líneas 12 a 17: Constructor con todos los datos.

```

18 |     public int getId() {
19 |         return id;
20 |     }
21 |
22 |     public void setId(int id) {
23 |         this.id = id;
24 |     }
25 |
26 |     public String getNombre() {
27 |         return nombre;
28 |     }
29 |
30 |     public void setNombre(String nombre) {
31 |         this.nombre = nombre;
32 |     }
33 |
34 |     public double getPrecio() {
35 |         return precio;
36 |     }
37 |
38 |     public void setPrecio(double precio) {
39 |         this.precio = precio;
40 |     }
41 | }

```

Líneas 18 a 41: Getters y setters de cada atributo (encapsulamiento).

022 - Clase 16-09 / src / main / java / edu / usal / model / ProductosDAO.java

Qué es y para qué sirve: DAO de productos con una **lista en memoria** (simula la base de datos).

Cómo defenderlo / qué decir: Es la implementación inicial «mock»: permite desarrollar el servlet y la vista antes de conectar una base real. Cuando se agregue JDBC, solo cambia esta clase (o se crea otra implementación elegida por una Factory) y el servlet no se modifica. Mejora posible: que sea una interfaz con implementaciones.

```

1 | package edu.usal.model;
2 |

```

Líneas 1 a 2: Paquete del modelo.

```

3 | import java.util.ArrayList;
4 | import java.util.List;
5 |

```

Líneas 3 a 5: Importaciones: **ArrayList**, **List**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

6 | public class ProductosDAO {
7 |
8 |     private List<Productos> listaProductos;
9 |

```

Líneas 6 a 9: Atributo privado: la lista de productos.

```

10 |     public ProductosDAO() {
11 |         this.listaProductos = new ArrayList<>();
12 |         this.listaProductos.add(new Productos(1, "Placa de Video", 420000));
13 |         this.listaProductos.add(new Productos(2, "Procesador 4 nucleos", 280000.50));
14 |         this.listaProductos.add(new Productos(3, "Memoria Ram 16GB", 210500));
15 |
16 |     }
17 |

```

Líneas 10 a 17: Constructor: crea la lista y la **carga con tres productos de ejemplo** (placa de video, procesador y memoria RAM).

```

18 |     public List<Productos> getAll(){
19 |         return this.listaProductos;
20 |     }
21 | }

```

Líneas 18 a 21: Devuelve todos los productos (equivale a un SELECT *).

022 - Clase 16-09 / src / main / java / edu / usal / servlet / PrimerServlet.java

Qué es y para qué sirve: Primer servlet: responde un «Hola Mundo» escribiendo el HTML con **PrintWriter**. Se declara en **web.xml**.

Cómo defenderlo / qué decir: Muestra lo esencial: extender **HttpServlet**, sobrescribir **doGet**, fijar el **tipo MIME** y escribir con **getWriter()**. Escribir HTML con **println** desde Java es lo que luego se evita delegando la presentación a las JSP (MVC). Dos importaciones (**HttpSession** y **HttpJspPage**) no se usan. Como **doPost** llama a **super.doPost**, un POST recibe el error por defecto (método no permitido).

```
1 | package edu.usal.servlet;
2 |
```

Líneas 1 a 2: Paquete de los servlets.

```
3 | import javax.servlet.ServletException;
4 | import javax.servlet.http.HttpServlet;
5 | import javax.servlet.http.HttpServletRequest;
6 | import javax.servlet.http.HttpServletResponse;
7 | import javax.servlet.http.HttpSession;
8 | import javax.servlet.jsp.HttpJspPage;
9 | import java.io.IOException;
10 | import java.io.PrintWriter;
11 |
```

Líneas 3 a 11: Importaciones: **ServletException**, **HttpServlet**, **HttpServletRequest**, **HttpServletResponse**, **HttpSession**, **HttpJspPage**, **IOException**, **PrintWriter**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```
12 | public class PrimerServlet extends HttpServlet {
13 |
```

Líneas 12 a 13: La clase **extiende HttpServlet**: así se convierte en un servlet.

```
14 |     @Override
15 |     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
```

Líneas 14 a 15: **@Override** + **doGet**: se ejecuta cuando llega una petición GET (escribir la URL en el navegador es un GET). Recibe req (petición) y resp (respuesta) y declara **ServletException** e **IOException**.

```
16 |         resp.setContentType("text/html;charset=UTF-8");
17 |
```

Líneas 16 a 17: Define el **tipo MIME** de la respuesta: text/html con codificación UTF-8 (tildes correctas).

```
18 |         PrintWriter out = resp.getWriter();
19 |
```

Líneas 18 a 19: Obtiene el **PrintWriter** para escribir el cuerpo de la respuesta.

```
20 |             out.println("<html>");
21 |             out.println("<head> <title>Servlet 16-09</title>");
22 |             out.println("</head>");
23 |             out.println("<body>");
24 |             out.println("<h1> Hola Mundo desde el Servlet! </h1>");
25 |             out.println("</body>");
26 |             out.println("</html>");
27 |
28 |         }
29 |
30 |     @Override
```

Líneas 20 a 30: Escribe línea por línea el documento HTML: html, head con título, body con un h1 «Hola Mundo desde el Servlet!». Lo que se escribe es lo que recibe el navegador.

```
31 |         protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
32 |             super.doPost(req, resp);
33 |         }
34 |
35 |     }
```

Líneas 31 a 35: **doPost** (peticiones POST): se limita a llamar al método de la clase padre, que responde con un error de método no permitido. O sea: este servlet solo acepta GET.

022 - Clase 16-09 / src / main / java / edu / usal / servlet / SegundoServlet.java

Qué es y para qué sirve: Servlet equivalente al anterior pero declarado con la **anotación @WebServlet** (sin tocar web.xml).

Cómo defenderlo / qué decir: Demuestra la configuración por anotaciones: **name** es el nombre del servlet y **urlPatterns** la URL (/segundo-servlet). Ventaja sobre web.xml: la URL está junto al código y no hay que editar otro archivo. Responde directamente con `resp.getWriter().println` (sin `setContentType`, usa el tipo por defecto).

```
1 | package edu.usal.servlet;  
2 |
```

Líneas 1 a 2: Paquete de los servlets.

```
3 | import javax.servlet.ServletException;  
4 | import javax.servlet.annotation.WebServlet;  
5 | import javax.servlet.http.HttpServlet;  
6 | import javax.servlet.http.HttpServletRequest;  
7 | import javax.servlet.http.HttpServletResponse;  
8 | import java.io.IOException;  
9 |
```

Líneas 3 a 9: Importaciones: **ServletException**, **WebServlet**, **HttpServlet**, **HttpServletRequest**, **HttpServletResponse**, **IOException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```
10 | @WebServlet(  
11 |     name = "servlet-anotaciones",  
12 |     urlPatterns = "/segundo-servlet"  
13 | )
```

Líneas 10 a 13: **Anotación @WebServlet:** registra el servlet sin web.xml, con nombre «servlet-anotaciones» y la URL **/segundo-servlet**.

```
14 | public class SegundoServlet extends HttpServlet {  
15 |  
16 |     @Override
```

Líneas 14 a 16: Extiende **HttpServlet**.

```
17 |     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {  
18 |         resp.getWriter().println("Hola Mundo desde el Servlet con Anotaciones!");  
19 |     }  
20 |  
21 |     @Override
```

Líneas 17 a 21: **doGet:** escribe directamente un texto en la respuesta con `getWriter().println`.

```
22 |     protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {  
23 |         super.doPost(req, resp);  
24 |     }  
25 |  
26 | }
```

Líneas 22 a 26: **doPost:** delega en el padre (error por defecto).

022 - Clase 16-09 / src / main / java / edu / usal / servlet / ProductosServlet.java

Qué es y para qué sirve: **Controlador** del patrón MVC: obtiene los productos del DAO y los muestra en una JSP mediante **forward**.

Cómo defenderlo / qué decir: Es el modelo más cercano a lo que se pide en el parcial. Puntos: **init()** crea el DAO **una sola vez** (ciclo de vida; como el DAO no tiene estado por usuario, es seguro compartirlo entre hilos); **doGet** consulta, **setAttribute** deja los datos en el request y **forward** transfiere a la vista dentro de WEB-INF (el usuario no puede abrirla directo). La línea comentada con **sendRedirect** no funcionaría: la JSP está en WEB-INF y además un redirect perdería el atributo del request.

```
1 | package edu.usal.servlet;
2 |
```

Líneas 1 a 2: Paquete de los servlets.

```
3 | import edu.usal.model.Productos;
4 | import edu.usal.model.ProductosDAO;
5 |
6 | import javax.servlet.RequestDispatcher;
7 | import javax.servlet.ServletException;
8 | import javax.servlet.annotation.WebServlet;
9 | import javax.servlet.http.HttpServlet;
10 | import javax.servlet.http.HttpServletRequest;
11 | import javax.servlet.http.HttpServletResponse;
12 | import java.io.IOException;
13 | import java.util.ArrayList;
14 | import java.util.List;
15 |
```

Líneas 3 a 15: Importaciones: **Productos**, **ProductosDAO**, **RequestDispatcher**, **ServletException**, **WebServlet**, **HttpServlet**, **HttpServletRequest**, **HttpServletResponse**, **IOException**, **ArrayList**, **List**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```
16 | @WebServlet(urlPatterns = "/productos")
```

Línea 16: Registra el servlet en la URL **/productos** con la anotación.

```
17 | public class ProductosServlet extends HttpServlet {
18 |
19 |     private ProductosDAO productosDAO;
20 |
21 |     @Override
```

Líneas 17 a 21: Extiende **HttpServlet**. Atributo privado: el **DAO** (el «modelo»).

```
22 |     public void init() throws ServletException {
23 |         this.productosDAO = new ProductosDAO();
24 |     }
25 |
26 |     @Override
```

Líneas 22 a 26: **init()**: método del ciclo de vida que se ejecuta una vez al crear el servlet; aquí instancia el DAO.

```
27 |     protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
28 |         super.doPost(req, resp);
29 |     }
30 |
31 |     @Override
```

Líneas 27 a 31: **doPost** delega en el padre (este servlet solo atiende GET).

```
32 |     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
33 |         List<Productos> productos = this.productosDAO.getAll();
34 |
```

Líneas 32 a 34: **doGet**: pide la lista de productos al DAO (**getAll()**).

```
35 |         req.setAttribute("listado", productos);
36 |
```

Líneas 35 a 36: **request.setAttribute("listado", productos)**: guarda la lista en el request con el nombre «listado»; la JSP la leerá con **`\${requestScope.listado}**».

```
37 |         RequestDispatcher dispatcher = req.getRequestDispatcher("/WEB-INF/productos/lista-productos.jsp");
38 |         dispatcher.forward(req, resp);
39 |
40 |         //resp.sendRedirect("lista-productos.jsp");
41 |     }
42 | }
43 |
```

Líneas 37 a 43: Obtiene el **RequestDispatcher** de la vista y hace **forward(req, resp)**: el control pasa a la JSP con el mismo request. La carpeta **WEB-INF** protege la vista de accesos directos. La línea comentada con **sendRedirect** es la alternativa (incorrecta aquí).

022 - Clase 16-09 / src / main / webapp / index.jsp

Qué es y para qué sirve: Página de inicio mínima de la aplicación (título «JSP Clase 16-09»).

Cómo defenderlo / qué decir: Es la welcome page: lo que se ve al abrir `http://localhost:8080/clase-16-09/`. No tiene lógica.

```
1 | <%--
2 |   Created by IntelliJ IDEA.
3 |   User: usal
4 |   Date: 16/9/26
5 |   Time: 7:27 p. m.
6 |   To change this template use File | Settings | File Templates.
7 | --%>
```

Líneas 1 a 7: Comentario JSP generado por el IDE (autor y fecha); no llega al navegador.

```
8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
```

Línea 8: Directiva page (UTF-8).

```
9 | <html>
10 | <head>
11 |   <title>Clase 16-09</title>
12 | </head>
13 | <body>
14 |   <h1>JSP Clase 16-09</h1>
15 |
16 | </body>
17 | </html>
```

Líneas 9 a 17: HTML básico: head con el título y body con un título h1.

022 - Clase 16-09 / src / main / webapp / WEB-INF / productos / lista-productos.jsp

Qué es y para qué sirve: **Vista** del patrón MVC: muestra la tabla de productos que dejó el servlet en el request, usando JSTL.

Cómo defenderlo / qué decir: Es una vista limpia: **cero código Java**, solo JSTL y EL. Lee la lista con `${requestScope.listado}` (alcance explícito de request) y contempla el caso de lista vacía con **c:choose**. Está dentro de **WEB-INF**, por lo que solo se llega a ella a través del servlet. Detalle: usa clases de Bootstrap sin cargar la hoja de estilos (no se ven los estilos).

```
1 | <%--
2 |   Created by IntelliJ IDEA.
3 |   User: usal
4 |   Date: 16/9/26
5 |   Time: 7:51 p. m.
6 |   To change this template use File | Settings | File Templates.
7 | --%>
```

Líneas 1 a 7: Comentario JSP generado por el IDE (autor y fecha); no llega al navegador.

```
8 | <%@ page contentType="text/html; charset=UTF-8" language="java" %>
```

Línea 8: Directiva page (UTF-8).

```
9 | <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
```

Línea 9: Directiva **taglib**: habilita la librería **core** de JSTL con el prefijo c.

```
10 | <html>
11 | <head>
12 |   <title>Lista de productos</title>
13 |   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
14 |         integrity="sha384-sRl14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNwLzeN8tE5YBujZqJLB" crossorigin="anonymous">
15 | </head>
16 | <body>
17 |   <h1>Listado de productos</h1>
```

Líneas 10 a 17: html, head con título y body con el título «Listado de productos».

```

18 | <c:choose>
19 |   <c:when test="{empty requestScope.listado}">
20 |     <div class="alert alert-info">
21 |       <p>No existe productos cargados en la BD</p>
22 |     </div>
23 |   </c:when>

```

Líneas 18 a 23: c:choose: estructura if / else. **c:when** con `{empty requestScope.listado}`: si el atributo «listado» no existe o está vacío, muestra el aviso «No existe productos cargados».

```

24 |   <c:otherwise>
25 |     <div class="container">
26 |       <table class="table table-striped table-hover bg-white shadow-sm border">
27 |         <thead class="table-dark">
28 |           <tr>
29 |             <td>ID</td>
30 |             <td>Nombre</td>
31 |             <td>Precio</td>
32 |           </tr>
33 |         </thead>
34 |         <tbody>

```

Líneas 24 a 34: c:otherwise (else): si hay datos, arma la tabla con su encabezado (ID, Nombre y Precio).

```

35 |           <c:forEach var="produ" items="{requestScope.listado}">
36 |             <tr>
37 |               <td>${produ.id}</td>
38 |               <td>${produ.nombre}</td>
39 |               <td>${produ.precio}</td>
40 |             </tr>

```

Líneas 35 a 40: c:forEach: recorre `{requestScope.listado}`; en cada vuelta **produ** es un producto y `{produ.id}`, `{produ.nombre}` y `{produ.precio}` llaman a sus getters. Una fila por producto.

```

41 |           </c:forEach>
42 |         </tbody>
43 |       </table>
44 |     </div>
45 |   </c:otherwise>
46 | </c:choose>
47 |
48 | </body>
49 | </html>
50 |

```

Líneas 41 a 50: Cierra el bucle, la tabla y el choose; cierra body y html.

Resumen para estudiar

- Servlet = clase que extiende **HttpServlet** y sobrescribe doGet/doPost. Recibe **request** y **response**.
- Ciclo de vida: **init()** (una vez) → **service()** → doGet/doPost (cada petición) → **destroy()** (una vez).
- Una instancia por servlet, un hilo por petición: no guardar datos de usuario en atributos del servlet.
- Mapeo: **web.xml** (servlet + servlet-mapping) o **@WebServlet**.
- **forward**: interno, conserva el request, la URL no cambia. **sendRedirect**: 302, el navegador hace otra petición, se pierde el request.
- MVC: Servlet (controlador) + DAO (modelo) + JSP/JSTL (vista en WEB-INF). Datos al JSP con **request.setAttribute**.
- Tipo MIME con **setContentType**; códigos 200/302/400/403/404/500.

Cómo se aplica en el parcial de logística

- Los servlets **LoginServlet** y **LogoutServlet** están en web.xml; **CamionServlet**, **ChoferServlet**, **ViajeServlet**, **MisViajesServlet** e **InicioServlet** usan **@WebServlet**: se cumple el punto «Servlets por anotaciones y web.xml».
- Todos siguen el patrón de ProductosServlet: consulta al DAO → **setAttribute** → **forward** a una JSP de WEB-INF; y tras un POST se usa **sendRedirect** (Post/Redirect/Get).

Preguntas típicas

¿Cuántas instancias de un servlet hay?

Una sola; cada petición se atiende en un hilo distinto sobre esa instancia. Por eso no se guardan datos del usuario en atributos del servlet.

¿Cuándo uso forward y cuándo sendRedirect?

forward para mostrar una vista con los datos que preparó el servlet; **sendRedirect** después de un POST (o para ir a otra URL) para que al refrescar no se repita la operación.

¿Por qué las JSP van en WEB-INF?

Porque WEB-INF no es accesible desde el navegador: la única forma de verlas es pasando por un servlet, que garantiza que los datos estén cargados y que se aplique la seguridad.